Modèle E2
Segmentation de l'anneau et localisation 2D / 3D du cerceau

Résumé complet : données, paramètres, architecture du CNN, entraînement, meilleure époque, passage du masque aux coordonnées 2D puis 3D

Partie vision du projet de vol autonome d'un drone Tello
Modèle : logs/E2_segmentation480_0 · code : ~/Documents/tello/tellovision

Fiche d'identité

Élément	Valeur
Nom de l'expérience	E2_segmentation480 (dossier logs/E2_segmentation480_0)
Tâche	Segmentation de l'anneau + présence, puis ajustement d'ellipse et pose 3D
Modèle	RingSegNet : encodeur MobileNetV3-small pré-entraîné (ImageNet) + décodeur U-Net
Entrée / sortie	Image 480×360 → masque 240×180 + probabilité de présence
Paramètres	1,67 million (encodeur 0,93 M, décodeur 0,74 M, tête de présence 577)
Données	615 images : 495 d'entraînement (284 cerceaux, 211 vides), 120 de validation (90 / 30)
Meilleure époque	**25** (sur 38 effectuées, arrêt anticipé à l'époque 37)
Perte de validation retenue	0,7016
Présence (validation)	exactitude 95,0 %, précision 98,8 %, rappel 94,4 %, F1 0,966, AP 0,998
Boîte englobante (validation)	IoU médiane 0,91 ; 83 % des images à IoU ≥ 0,5 ; AP@0,5 = 0,81
Centre 2D (validation)	médiane 9,2 px (cage : 3,9 px) ; moyenne 21,9 px
Distance 3D (cage)	erreur médiane 1,6 % (3 cm) entre 0,9 et 3,1 m
Temps de calcul (CPU)	64 ms par image : CNN 42 ms + ellipse 22 ms + pose 0,4 ms (≈ 15 images/s)

1. Pourquoi la segmentation

Un détecteur doit répondre à deux questions : **y a-t-il un cerceau ?** (classification) et **où est-il exactement ?** (localisation). Les modèles précédents (E0, E1) répondaient aux deux avec une seule couche dense qui sortait 6 nombres pour toute l'image : c'est la **régression globale**. Elle classe bien, mais localise mal avec peu d'exemples : chaque position de l'image a ses propres poids, et quand le réseau hésite il répond une position « moyenne ».

E2 sépare les deux questions. La **classification** reste globale (une probabilité par image). La **localisation** devient une prédiction **pixel par pixel** : le réseau colorie les pixels du tube, avec les mêmes poids partout dans l'image, puis une étape géométrique exacte ajuste l'ellipse sur ces pixels.

Modèle (même validation stable)	Présence	Centre médian	Grand axe médian	B/A
E0 – régression 320×240	89,2 %	68 px	16,6 %	0,082
E1 – régression 480×360	90,8 %	78 px	28,1 %	0,086
E2 – segmentation 480×360	**95,0 %**	**9,2 px**	**3,9 %**	**0,083**

2. Les données

2.1 Jeu de données et découpage

615 images de la caméra du Tello (960×720), prises dans le couloir (cerceau gris) et dans la cage (cerceau violet de 68 cm) : 374 avec cerceau, 241 vides. Règle d'annotation : si moins d'un tiers du cerceau est visible, l'image est vide. Chaque cerceau est annoté par une ellipse (centre, grand axe, petit axe, inclinaison), qui peut déborder de l'image.

Le découpage entraînement / validation est **stable** : chaque image appartient à une tranche de 5 secondes (d'après l'horodatage de son nom), et le côté de la tranche est fixé par un hachage (CRC32) de son numéro avec la graine 0. Les images quasi identiques d'une même rafale vont donc toutes du même côté, et réétiqueter une image ne déplace jamais les autres. Résultat : 495 images d'entraînement (284 cerceaux, 211 vides) et 120 de validation (90 cerceaux, 30 vides), dans les deux lieux.

2.2 Prétraitement

- redimensionnement en **480×360** (rapport 4:3 conservé) ;
- passage de BGR (OpenCV) à RGB, valeurs ramenées dans [0, 1] puis normalisées avec la moyenne (0,485 ; 0,456 ; 0,406) et l'écart-type (0,229 ; 0,224 ; 0,225) d'ImageNet, comme lors du pré-entraînement ;
- tenseur au format (canaux, hauteur, largeur) : un lot de 16 images est un tenseur (16, 3, 360, 480).

2.3 Le masque cible

Le réseau apprend à reproduire un **masque de l'anneau**, dessiné automatiquement à partir de l'ellipse annotée (aucune annotation supplémentaire) : l'ellipse est ramenée à la résolution du masque (240×180, soit un pas de 2) et tracée avec une épaisseur de **4 % du diamètre** (tube de 2,5 cm sur 68 cm), au minimum 2 pixels. Une image vide a un masque entièrement noir ; une « corde » (arc trop court) a un masque inconnu, ignoré par la perte.

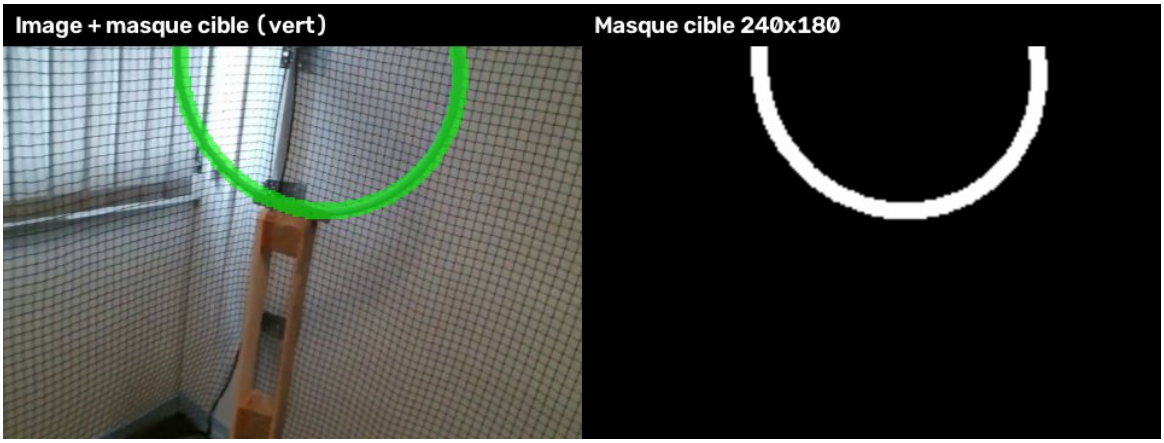


Figure 1 – Masque cible dessiné à partir de l'ellipse annotée.

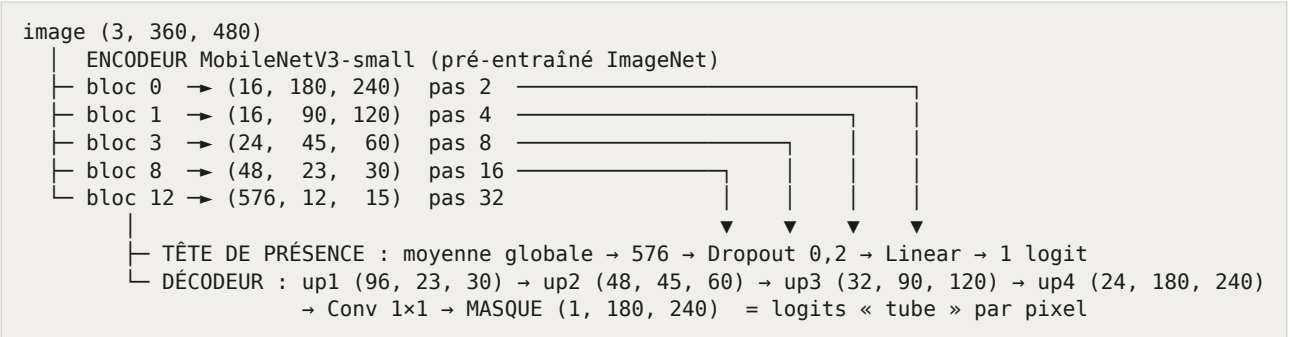
2.4 Augmentations (entraînement seulement)

Transformation	Probabilité	Paramètres	Rôle
Miroir horizontal	0,5	—	Cerceau vu de l'autre côté
Rotation de caméra H = K R K^{-1}	0,6	lacet ±12°, tangage ±8°, roulis ±12° ; K de la calibration	Le drone tourne ; ellipse transformée exactement
Zoom + décalage	0,5	échelle 0,6 à 1,3 ; décalage ±15 %	Distance et position variées
Visibilité minimale	—	au moins 34 % du contour dans l'image	Cohérent avec la règle du tiers
Couleur / lumière	0,8	luminosité ±35 %, contraste ±30 %, saturation ±40 %, teinte ±4	Contre-jour, exposition
Flou de bougé	0,3	noyau ≤ 11 px	Mouvement du drone
Bruit	0,3	écart-type ≤ 8	Capteur
Rectangle masqué	0,3	côté ≤ 15 % de l'image	Occultation
Compression JPEG	0,5	qualité 35 à 80	Flux Wi-Fi
Collage sur un autre fond	0	désactivé	Créait des anneaux « fantômes »

Chaque image est vue une fois par époque (répétitions 1 / 1), avec un tirage différent à chaque époque.

3. Le réseau RingSegNet

Le réseau a la forme d'un U : un **encodeur** réduit progressivement l'image en cartes de caractéristiques de plus en plus abstraites et de moins en moins résolues ; un **décodeur** remonte la résolution en réutilisant les cartes intermédiaires de l'encodeur (**connexions de saut**) pour retrouver le détail fin du tube. Deux têtes répondent aux deux questions.



3.1 L'encodeur : MobileNetV3-small

C'est un réseau conçu pour les téléphones : peu de calculs, mais pré-entraîné sur 1,2 million d'images (ImageNet), il sait déjà reconnaître bords, textures et formes. Ses 13 blocs sont faits de :

- **une convolution d'entrée** 3×3 de pas 2 (16 filtres), suivie d'une normalisation par lot et de l'activation **hard-swish** (une version rapide de x·sigmoïde(x)) ;
- **blocs « résiduels inversés »** : une convolution 1×1 qui augmente le nombre de canaux, une convolution **en profondeur** 3×3 ou 5×5 (un filtre par canal, beaucoup moins coûteuse qu'une convolution classique), un module **squeeze-and-excitation** (moyenne globale de chaque canal → petite couche dense → poids qui renforcent les canaux utiles), une convolution 1×1 de projection, et une connexion résiduelle quand les dimensions le permettent ;

- **une convolution finale** 1×1 vers 576 canaux (hard-swish).

La résolution est divisée par 2 cinq fois, toujours par des **convolutions de pas 2** (pas de couche de max-pooling). On prélève les sorties des blocs 0, 1, 3, 8 et 12, aux pas 2, 4, 8, 16 et 32.

3.2 Le décodeur U-Net

Chaque bloc de remontée (UpBlock) : interpolation bilinéaire jusqu'à la taille de la carte de l'encodeur correspondante, concaténation avec elle, puis deux fois (convolution 3×3 + normalisation par lot + ReLU). Une dernière convolution 1×1 produit un logit par pixel.

Bloc	Entrées (canaux)	Sortie	Paramètres
up1	576 (pas 32) + 48 (pas 16)	(96, 23, 30)	622 k
up2	96 + 24 (pas 8)	(48, 45, 60)	73 k
up3	48 + 16 (pas 4)	(32, 90, 120)	28 k
up4	32 + 16 (pas 2)	(24, 180, 240)	16 k
Tête du masque	Conv 1×1, 24 → 1	(1, 180, 240)	25

3.3 La tête de présence (classification)

Moyenne globale (global average pooling) de la carte la plus profonde (576×12×15 → 576 valeurs), Dropout de 0,2, puis une couche dense vers 1 logit ; la probabilité de présence est la sigmoïde de ce logit. Ici la moyenne globale est le bon choix : la présence est une question globale, la position n'importe pas. Pour la localisation au contraire, on garde toute l'information spatiale grâce au décodeur.

Où	Opération	Rôle
Encodeur (descente)	Convolutions de pas 2	Réduire la résolution, agrandir le champ réceptif
Modules squeeze-and-excitation	Moyenne globale par canal	Pondérer les canaux
Décodeur (remontée)	Interpolation bilinéaire + connexions de saut	Retrouver le détail fin
Tête de présence	Moyenne globale + Dropout 0,2	Décision globale, limiter le surapprentissage
Partout	Normalisation par lot	Stabiliser l'apprentissage

4. La perte

```
perte = BCE(présence)                  poids des positifs : 1
      + 1 × BCE(masque) pondérée       poids des pixels « tube » : 10
      + 1 × Dice(masque)               images contenant un anneau seulement
(perte de masque ignorée pour les « cordes », dont le masque est inconnu)
```

L'anneau n'occupe que quelques pour cent des pixels. Avec une entropie croisée simple, prédire « fond » partout donnerait déjà une perte faible : on multiplie donc par 10 le poids des pixels du tube. La perte de **Dice** mesure directement le recouvrement entre masque prédit P et masque vrai T :

$$\text{Dice} = 1 - (2 \cdot \Sigma\, P \cdot T + 1) / (\Sigma\, P + \Sigma\, T + 1)$$

Elle vaut 0 pour un recouvrement parfait et ne dépend pas du nombre de pixels de fond.

5. L'entraînement

Réglage	Valeur
Optimiseur	AdamW, pas 10^{-3} pour le décodeur et les têtes, 10^{-4} pour l'encodeur (× 0,1), pénalité des poids (weight decay) 10^{-4}
Calendrier du pas	Cosinus sur 50 époques : pas = pas initial × (1 + cos(π·époque/50)) / 2 ; à l'époque 25 il vaut la moitié du pas initial
Lot	16 images
Nombre d'époques	50 au maximum ; arrêt anticipé après 12 époques sans amélioration
Choix du meilleur modèle	Plus petite perte de validation
Graine aléatoire	0
Matériel	CPU, environ 80 s par époque, 51 minutes au total

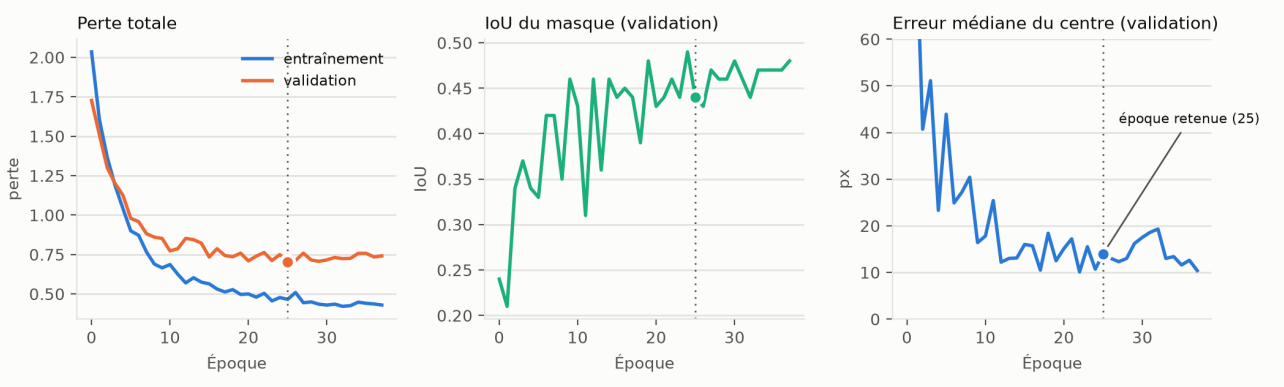


Figure 2 – Courbes d'apprentissage. Les pointillés marquent l'époque retenue (25). Les erreurs de centre de ce graphique sont calculées avec la première version du passage masque → ellipse (§7).

Meilleure époque : 25, avec une perte de validation de 0,7016. Après elle, la perte de validation stagne (0,70 à 0,76) pendant que la perte d'entraînement continue de baisser : le réseau commence à sur-apprendre, et l'arrêt anticipé s'est déclenché à l'époque 37. Remarque : le critère de choix est la perte, pas l'erreur du centre ; certaines époques suivantes avaient un centre médian un peu meilleur (10 px). Pour la localisation, choisir sur l'erreur du centre serait plus pertinent.

6. Métriques et résultats (validation stable, 120 images)

Toutes les mesures portent sur les **120 images de validation** (90 avec cerceau, 30 sans), jamais vues pendant l'entraînement et séparées par tranches de 5 s pour éviter les quasi-doublons (§2.1). Le modèle doit répondre à trois questions, chacune avec ses métriques : **y a-t-il un cerceau ?** (classification), **où est-il dans l'image ?** (boîte, ellipse, centre) et **où est-il dans l'espace ?** (§8.5).

6.1 Définitions des métriques

Présence (classification). Le réseau donne une probabilité p ; on décide « cerceau » si p ≥ 0,5. En comparant à la vérité, chaque image tombe dans une des quatre cases de la **matrice de confusion** :

Case	Signification	Conséquence pour le drone
VP (vrai positif)	cerceau présent, détecté	—

Case	Signification	Conséquence pour le drone
FN (faux négatif)	cerceau présent, **raté**	le drone continue à chercher : perte de temps
FP (faux positif)	**rien**, mais cerceau annoncé	le drone vise un objet qui n'existe pas : dangereux
VN (vrai négatif)	rien, et rien annoncé	—

- **Exactitude** = (VP + VN) / total : part des images bien classées. Trompeuse si les classes sont déséquilibrées (ici 90 cerceaux pour 30 vides).
- **Précision** = VP / (VP + FP) : quand le modèle annonce un cerceau, à quel point peut-on le croire ? C'est la métrique de **sécurité** (faux positifs).
- **Rappel** (sensibilité) = VP / (VP + FN) : parmi les vrais cerceaux, combien sont trouvés ?
- **Spécificité** = VN / (VN + FP) : parmi les images vides, combien sont reconnues vides ?
- **F1** = 2 · précision · rappel / (précision + rappel) : moyenne harmonique, élevée seulement si les deux le sont.
- **Courbe précision / rappel et AP** (average precision) : en faisant varier le seuil de 0 à 1, on trace la précision en fonction du rappel ; l'AP est l'aire sous cette courbe (1 = parfait). Elle juge le modèle indépendamment du seuil choisi.

Localisation (régression géométrique). Le recouvrement entre la forme prédite et la forme de référence se mesure par l'**IoU** (Intersection over Union) = aire commune / aire totale couverte par les deux (1 = identiques, 0 = disjointes). On le calcule sur trois objets :

- **IoU du masque** : pixels « tube » prédits et vrais. Sévère pour un anneau fin : un décalage de 2 px sur un tube de 4 px divise déjà l'IoU par deux, même si l'ellipse finale est excellente.
- **IoU de la boîte englobante** : le rectangle qui contient le cerceau (§6.4), la métrique standard des détecteurs (YOLO, Faster R-CNN).
- **IoU de l'ellipse pleine** : disque elliptique prédit et vrai, y compris la partie hors image.

S'y ajoutent les erreurs directement utiles au vol : **erreur du centre** (pixels), **erreur relative du grand axe** (elle se transmet telle quelle à la distance), **erreur du rapport B/A** (qui fixe l'orientation du cerceau) et **erreur d'inclinaison** de l'ellipse (degrés).

6.2 Présence

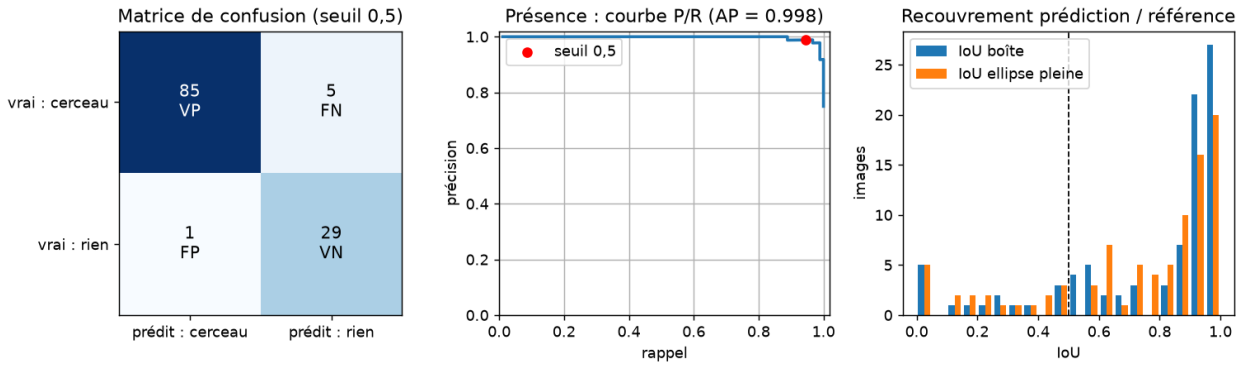


Figure 3 – Gauche : matrice de confusion au seuil 0,5. Milieu : courbe précision / rappel de la présence (point rouge : seuil 0,5). Droite : distribution des IoU boîte et ellipse sur les 90 images avec cerceau (une image sans prédiction compte pour un IoU de 0).

Métrique	Valeur	Lecture
Matrice de confusion	VP 85 · FN 5 · FP 1 · VN 29	6 erreurs sur 120
Exactitude	95,0 %	114 images sur 120

Métrique	Valeur	Lecture
Précision	**98,8 %**	1 seule fausse alarme
Rappel	94,4 %	5 cerceaux ratés
Spécificité	96,7 %	29 images vides sur 30
F1	0,966	
AP (aire sous P/R)	**0,998**	les probabilités classent presque parfaitement les images

Le modèle est **prudent** : il préfère rater un cerceau (5 FN) que d'en inventer un (1 FP), ce qui est le bon compromis pour un drone. L'AP de 0,998 montre que les cerceaux ratés ont tout de même une probabilité plus haute que les images vides : un seuil un peu plus bas augmenterait le rappel presque sans fausse alarme. Pour le vol, la guidance exige en plus **3 images consécutives** avant d'agir, ce qui élimine les fausses alarmes isolées.

6.3 Localisation 2D

Mesure	Toutes images	Cage (compétition)	Couloir
IoU du masque	0,44	—	—
Centre 2D, médiane	9,2 px	**3,9 px**	23,9 px
Centre 2D, moyenne	21,9 px	12,9 px	33,5 px
Grand axe (≈ distance), médiane	3,9 %	**1,7 %**	13,7 %
Rapport B/A, erreur moyenne	0,083	**0,041**	0,139
Inclinaison (ellipses aplaties), médiane	—	9,5° (9 ellipses)	61,7° (17)

Dans la cage, lieu de la compétition, l'ellipse est trouvée à environ 4 pixels près. Les erreurs restantes viennent surtout des petits cerceaux gris du couloir, à contre-jour, qui ne seront pas en compétition. L'IoU du masque (0,44) paraît faible, mais il mesure la superposition d'un anneau de quelques pixels d'épaisseur : l'ellipse ajustée dessus, qui est ce qui sert, est bien plus précise (IoU médian 0,85, §6.4).

6.4 Boîtes englobantes

Une **boîte englobante** est le plus petit rectangle aligné sur les axes de l'image qui contient l'objet, décrit par ses coins (x1, y1, x2, y2). C'est la sortie des détecteurs classiques (YOLO, SSD, Faster R-CNN), et c'est ainsi que nos premières annotations et le tout premier modèle (HoopNet_0, rapport V2/V3) décrivaient le cerceau.

Pourquoi E2 ne prédit pas de boîte mais une ellipse. Une boîte ne dit pas comment le cerceau est orienté : un cerceau vu de face et le même cerceau tourné de 40° peuvent avoir presque la même boîte. Or la pose 3D (§8) a besoin de la forme exacte : le grand axe donne la distance, le rapport B/A et l'inclinaison donnent l'orientation de l'axe du cerceau. L'ellipse (5 nombres : centre, deux demi-axes, angle) contient toute l'information de la boîte, et davantage.

La boîte se déduit de l'ellipse (fonction `ellipse_bbox`) : pour une ellipse de centre (cx, cy), demi-axes A, B et angle θ, les demi-largeur et demi-hauteur de la boîte sont $w = \sqrt{(A^2\cos^2\theta + B^2\sin^2\theta)}$ et $h = \sqrt{(A^2\sin^2\theta + B^2\cos^2\theta)}$. On peut donc évaluer E2 avec les métriques des détecteurs, pour le comparer à des modèles qui prédisent des boîtes. La boîte de référence est déduite de la même façon de l'ellipse annotée, puis limitée au cadre de l'image (partie visible).

Métrique (90 images avec cerceau)	Boîte (partie visible)	Boîte amodale	Ellipse pleine
IoU moyen	0,76	0,72	0,72
IoU médian	**0,91**	0,85	0,85
Part des images à IoU ≥ 0,5	**83 %**	80 %	79 %
Part des images à IoU ≥ 0,75	66 %	63 %	61 %
AP@0,5 (style détecteur)	**0,81**	—	—
AP@0,75	0,60	—	—

- **AP@0,5** : la métrique classique des détecteurs (PASCAL VOC). Une détection compte comme juste si p ≥ seuil et IoU de boîte ≥ 0,5 ; on trie les détections par probabilité et on calcule l'aire sous la courbe précision / rappel. **AP@0,75** est la version exigeante (boîte très bien placée).
- **Boîte amodale** : boîte de l'ellipse entière, même la partie hors image. Elle est plus sévère que la boîte visible, car il faut deviner où passe le cerceau en dehors du cadre.
- Le pic de la distribution (figure 3, droite) est entre 0,9 et 1 : quand le cerceau est trouvé, sa boîte est presque parfaite. Les IoU faibles viennent des 5 cerceaux ratés (IoU 0) et de quelques cerceaux très coupés ou lointains du couloir.

À retenir. La boîte est une bonne métrique de **comparaison** avec les détecteurs du commerce (AP@0,5 = 0,81 ici), mais une mauvaise **sortie** pour notre problème : elle perd l'orientation du cerceau, indispensable pour le traverser. E2 prédit donc une ellipse, et la boîte n'est calculée que pour l'évaluation.

7. Du masque aux coordonnées 2D

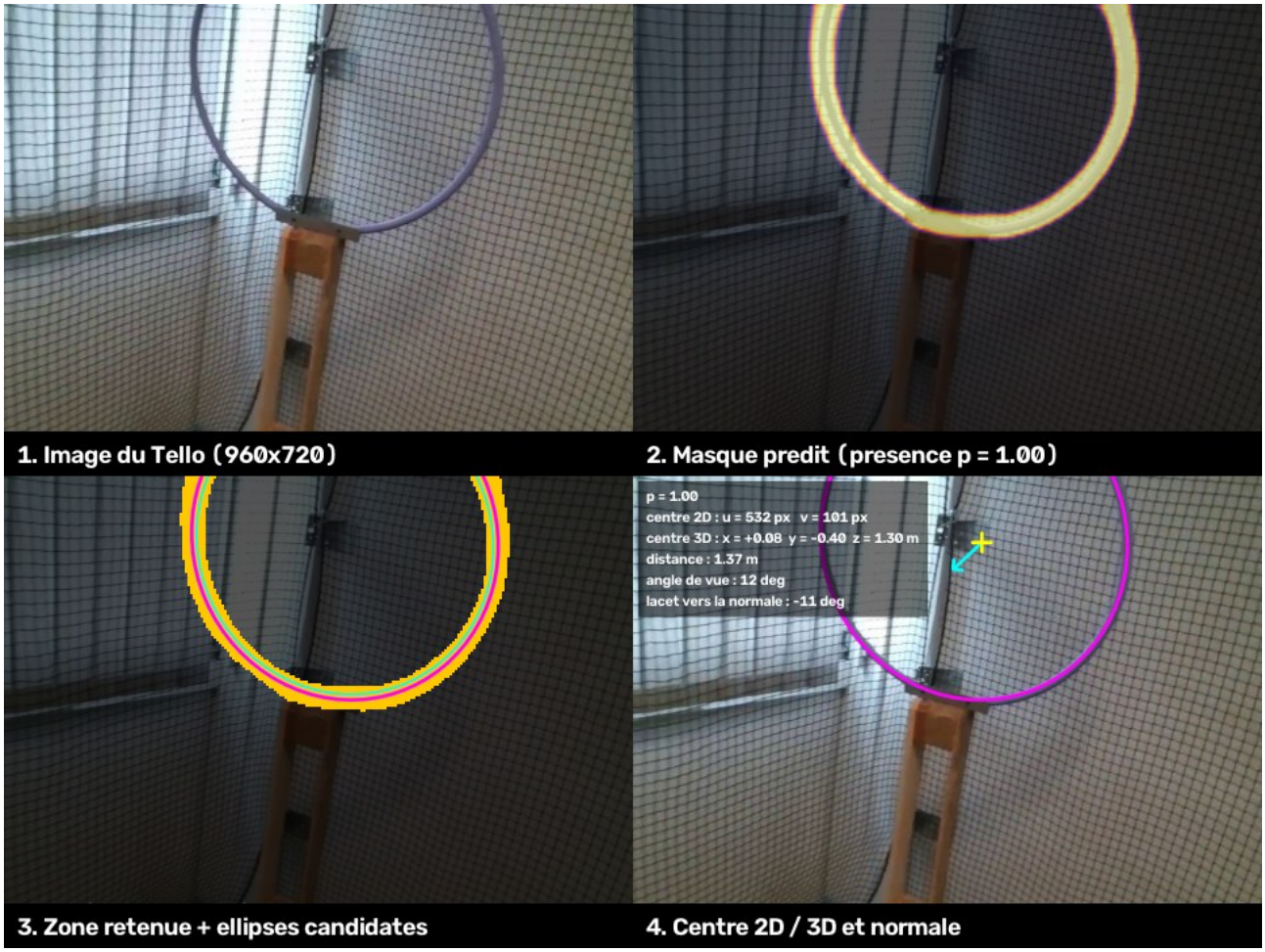


Figure 4 – La chaîne sur une image de la cage. 2 : probabilité « tube » prédite. 3 : zone retenue (jaune) et les deux ellipses candidates (A en magenta, B en cyan), ici confondues. 4 : résultat final.

7.1 Ajuster l'ellipse sur le masque

- **Seuil** : on garde les pixels de probabilité ≥ 0,5.
- **Plus grande zone connexe** seulement : les petites taches isolées sont du bruit.
- **Candidat A – RANSAC** : 150 fois, on tire 5 pixels au hasard, on ajuste une ellipse (cv2.fitEllipse) et on compte les pixels à moins de 4 % de son contour ; on garde la meilleure et on la réajuste sur tous ses pixels proches. Efficace quand le masque est un anneau ou un arc.
- **Candidat B – contour extérieur** : ellipse ajustée sur le bord extérieur de la zone, réduite d'une demi-épaisseur de tube. Efficace pour les petits cerceaux, dont le masque devient une tache pleine.
- **Choix** : la candidate qui passe sur les pixels les plus probables (probabilité moyenne sur 128 points de son contour).
- **Changement de repère** : du masque (240×180) à l'image (960×720), facteur 4 sur les coordonnées des centres de pixels.

Cette version a remplacé un RANSAC seul, qui échouait sur les taches pleines (ellipses 3 à 5 fois trop grandes) : centre médian de 14,0 à 9,2 px, grand axe moyen de 25,4 % à 13,2 %, sans réentraîner.

7.2 Le centre 2D du cerceau n'est pas le centre de l'ellipse

Par la perspective, le centre de l'ellipse n'est pas exactement l'image du centre du cerceau : la moitié la plus proche du cerceau paraît plus grande. Le centre 2D renvoyé (u, v) est donc la **projection du centre 3D** calculé au §8, par la calibration (distorsion comprise). L'écart est faible de loin et quand le drone est dans l'axe ; dans la validation de la cage il vaut 5,5 px en médiane et jusqu'à 48 px, de près et de biais.

8. Des coordonnées 2D à la 3D

8.1 La calibration de la caméra

Grandeur	Valeur	Sens
Focales fx, fy	927,1 et 925,8 px	Conversion angle ↔ pixels
Centre optique cx, cy	475,8 ; 386,3 px	Point de l'image dans l'axe de la caméra
Distorsion k1, k2, p1, p2, k3	−0,015 ; 0,035 ; 0,002 ; 0,002 ; 0,034	Très faible
Erreur de reprojection	0,23 px sur 39 images de damier	Très bonne calibration
Champ de vision	54,7° × 42,5° (65,9° en diagonale)	Le flux 960×720 n'utilise que le centre du capteur

Avant le calcul, la distorsion est corrigée : 72 points du contour de l'ellipse sont redressés (cv2.undistortPoints) et l'ellipse est réajustée.

8.2 Du cercle vu en ellipse au cercle 3D

Chaque point de l'ellipse correspond à un rayon qui part de la caméra : ces rayons forment un **cône**. Si C est la matrice de la conique de l'ellipse (en pixels) et K la matrice de la caméra, le cône a pour équation $X^T Q X = 0$, avec $Q = K^T C K$ (X en mètres, repère caméra). On diagonalise Q : valeurs propres $\lambda_1 \geq \lambda_2 > 0 > \lambda_3$, vecteurs propres rangés dans une matrice V. Seuls deux plans coupent ce cône selon un cercle de rayon r = 0,34 m :

```
a = √((λ1 − λ2) / (λ1 − λ3))        b = √((λ2 − λ3) / (λ1 − λ3))        s = ±1   (deux solutions)

normale  n = V · [ s·a,  0,  −b ]
centre   c = r / √(−λ1·λ3) · V · [ s·a·λ3,  0,  −b·λ1 ]

signes fixés pour que le cerceau soit devant la caméra (c_z > 0) et que la normale pointe
dans le sens de la traversée (n_z > 0)
```

Référence : Chen, Wu, Wada, « Camera calibration with two arbitrary coplanar circles », ECCV 2004. Vérification sur 500 cercles simulés avec la vraie calibration : erreur maximale de 0,4 mm sur le centre et de 0,13° sur la normale. Toute l'erreur finale vient donc du réseau, pas de la géométrie.

8.3 Les grandeurs renvoyées

Grandeur	Calcul	Utilisation
Centre 3D (x, y, z)	c, en mètres ; x à droite, y vers le BAS, z devant	Position du cerceau
Distance	$\|c\|$	Phase d'approche
Centre 2D (u, v)	projection de c par la calibration	Garder le cerceau centré
Normale n	vecteur unitaire, sens de la traversée	Direction d'entrée
Lacet vers la normale	$\text{atan2}(n_x, n_z)$	Rotation pour se placer face au cerceau
Angle de vue	$\text{acos}(n_z)$	Écart à l'axe du cerceau

8.4 L'ambiguïté double



Figure 5 – Frame 0459 : les deux solutions ont le même centre (0,99 m) mais deux orientations, tournées de +30° et −45°.

Une ellipse correspond à deux orientations possibles du cercle, comme une pièce tournée à gauche ou à droite. Règles de choix : (1) suivi d'une image à l'autre (la normale la plus proche de la précédente) ; (2) sinon, cerceau vertical, donc normale horizontale. Quand les deux restent proches, le résultat est marqué « ambigu ». L'ambiguïté disparaît quand le drone s'aligne : les deux solutions se rejoignent quand B/A = 1. Une convention sur l'orientation connue des cerceaux dans la salle, combinée au cap du drone, permettra un choix franc de loin.

8.5 Précision 3D

Référence : la pose calculée à partir de l'ellipse annotée, comparée à celle calculée à partir de l'ellipse de E2.

Mesure (médiane)	Cage (0,9 à 3,1 m)	Couloir (2,7 à 5,5 m)
Distance	**1,6 % (3 cm)**	15,6 % (44 cm)
Centre 3D	**4 cm**, dont 1,6 cm latéralement	44 cm

Mesure (médiane)	Cage (0,9 à 3,1 m)	Couloir (2,7 à 5,5 m)
Centre 2D (u, v)	**5,2 px**	21,2 px
Normale	6° (24° pour 10 % des cas)	32°

Contrôle indépendant : le centre du cerceau est à environ 1,20 + 0,34 = 1,54 m du sol. Sur une séquence de la cage, il est mesuré en médiane 0,37 m au-dessus de la caméra (y = −0,37 m), ce qui correspond à une caméra tenue vers 1,15 m.

Limites à connaître pour le guidage. près de l'axe du cerceau, B/A ≈ cos(angle de vue) varie très peu : une erreur de 0,04 sur B/A laisse l'angle entre 0 et environ 16°. En fin d'approche, il vaut mieux s'appuyer sur la position du centre (précise à quelques centimètres) que sur la normale. Enfin, le cerceau ne tient plus entièrement dans l'image en dessous d'environ 0,87 m : le dernier mètre se fait à l'aveugle.

9. Limites et suite

Point	Piste
Petits cerceaux gris du couloir mal mesurés	Hors compétition ; plus d'images si besoin
Meilleur modèle choisi sur la perte	Le choisir sur l'erreur du centre
Ambiguïté double de loin	Convention sur l'orientation des cerceaux + cap du drone
Mesures bruitées d'une image à l'autre (≈ 9 cm)	Filtrage temporel (moyenne glissante, Kalman)
Relier la vision au contrôle	guidance.py : phases approche / alignement / traversée, format des consignes à fixer avec la boucle du collègue

Annexe – Commandes et fichiers

```
cd ~/Documents/tello && source venv/bin/activate

# réentraîner E2
python -m tellovision.main configs/E2_segmentation480.yaml train
# coordonnées sur des images, avec affichage
python -m tellovision.detector "captures/hoop/*.jpg" --show
# séquence d'images -> vidéo + CSV ; ou flux du drone (sans décollage)
python tools/localize.py --images "captures/*/frame_*_17907*.jpg"
python tools/localize.py --tello
# comparer des modèles sur la même validation
python tools/compare_models.py logs/E0_regression320_0 logs/E2_segmentation480_0

tellovision/seg.py       masque cible, RingSegNet, perte, masque -> ellipse
tellovision/pose.py      ellipse -> cercle 3D (deux solutions, choix)
tellovision/detector.py  chaîne complète image -> centre 2D / 3D, affichage
configs/camera.yaml      calibration de la caméra
configs/hoop.yaml        diamètre du cerceau (0,68 m)
```